

Traditional User-Interface Graphics

Peter Occil

This version of the document is dated 2026-07-30.

This page discusses aspects of the traditional design (up to about the year 2003) of user-interface graphics, such as button and border styles, icons, and mouse pointers. User interfaces found in video games are outside the scope of this document.

1 Contents

- **Contents**
- **Display Modes**
- **Button and Border Styles**
 - **Colors for Button and Border Styling**
 - **Traditional Button Styles**
 - **Button and Border Drawing Challenge**
- **Icons and Cursors**
- **Drawing Style**
- **Animations**
- **Flexible User Interface Graphics**
- **Relevant Works**
- **Worthy Mentions**
- **License**
- **End Notes**

2 Display Modes

In this document:

- A *display mode* is a way to set up a computer display to show graphics.
- *Screen resolution* gives the number of columns and rows of pixels (point samples) the display mode can effectively show. For example, if a display mode's screen resolution is 320×200 , it can show 320 columns and 200 rows of pixels.¹
- *Pixel density* gives a display mode's horizontal and vertical pixel spacing, each in pixels per inch.²

Some display modes follow, along with their screen resolutions and commonly implemented pixel densities:

- VGA's (IBM Video Graphics Array) 640×480 16-color display mode: 96 horizontal and vertical pixels per inch.

¹This definition disregards whether a display shows an image using interlacing (for example, alternating between showing only odd rows and only even rows) or a progressive-scan method (all rows are displayed each frame).

²These two values for pixel density need not match the true pixel density of a particular computer display. For example, the drivers that come with Windows version 3.x employ a pixel density that is greater, on average, than the true one to aid readability of text (chapter 14 of Petzold 1990). Moreover, two display modes with the same screen resolution can differ in their pixel density, even if both are meant for displays with the same aspect ratio. For example, a 320×200 display mode can have 40 or 48 vertical pixels per inch, even if both are intended for displays with the 4:3 aspect ratio typical in 2000 and earlier.

- IBM Enhanced Graphics Adapter’s (EGA) 640×350 16-color display mode: 96 horizontal and 72 vertical pixels per inch.
- IBM Color/Graphics Adapter’s (CGA) 640×200 2-color display mode: 96 horizontal and 48 vertical pixels per inch.

An image can be adapted for display modes with pixel densities that differ from the VGA mode just given (which is the usual one in the mid-1990s) by scaling the image’s width, height, or both. For example, a 300×300 image, when adapted for the EGA mode, becomes a shrunk 300×225 image (the height becomes $72/96 = 3/4$ of the original height). If a display mode’s horizontal and vertical pixel spacing are equal, one way to find an appropriate scaling factor for images is to divide the mode’s pixels per inch by 96 (or whatever value of pixels per inch is the “normal” one); examples of such a factor are 1.25 (for the pixel density 1.25 to 1; IBM 8514/a), 2 (for high-pixel-density displays), and 3 (for very-high-pixel-density displays).

More generally, units similar to the spacing from one pixel to another may be employed as units of measure for user-interface elements, for design purposes to promote right-sized user interfaces. Examples include *dialog box units*³ (which depend on the font in which text is rendered) and *effective pixels*⁴ (which depend on the kind of display and its size, among other things).

3 Button and Border Styles

This section discusses the traditional drawing style for buttons and the borders around those buttons and other user-interface elements.

3.1 Colors for Button and Border Styling

Traditionally, the kinds of colors used to design buttons and borders in user interfaces (also called *system colors*) were the following. The given default values are those found in the “Windows Standard” color scheme in Windows 95.

- Button face color (by default, (192, 192, 192)).
- Window frame color (by default, (0,0,0) or black).
- Button highlight color (by default, (255,255,255) or white).⁵ Often, this is a lighter version of the button face color.
- Button shadow color (by default, (128, 128, 128)). Often, this is a darker version of the button face color.
- Button “light” highlight color (by default, (192, 192, 192)).⁶ Often, this is either the button face color or comes between the highlight and face colors.
- Button dark shadow color (by default, (0,0,0) or black).⁷ Often, this is either the window frame color or a darker version of the button shadow color.

User-interface conventions other than Windows 95 may use only a subset of these system colors or derive some of them (such as the highlight or shadow color) from a base color (such as the button face color).⁸

This discussion does not apply to black-and-white buttons (buttons with colors limited to black and white). For example, buttons in Mac OS versions before 8.0 and in Windows 1.x and 2.x were of that kind.

³<https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getdialogbaseunits>

⁴<https://learn.microsoft.com/en-us/windows-hardware/design/component-guidelines/guidance-for-rounded-display-bezels>

⁵In Windows, this color was first supported in version 3.1. In the Foundation Class Library’s implementation of toolbars, this color is fixed to (255,255,255); see MFC Technical Note TN031, “Control Bars”.

⁶In Windows, this color was first supported in the Win32 interface, and so was not available in Windows 3.1 or earlier.

⁷In Windows, this color was first supported in the Win32 interface, and so was not available in Windows 3.1 or earlier.

⁸Examples include the Motif toolkit (see the section “Worthy Mentions”) as well as **Windows 10 and Windows 11**. <https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getsyscolor>

3.2 Traditional Button Styles

The following appearances are traditionally seen in ordinary buttons:

- *Monochrome appearance*: The button’s text and icons are drawn using the button shadow color instead of its normal colors, but with transparency and opacity in the label preserved.
- *Embossed appearance*: The button’s text and icons are drawn—
 - using the button highlight color instead of its normal colors, then
 - using the button shadow color instead of its normal colors and offset 1 unit upward and 1 unit to the left,

in each case with transparency and opacity in the label preserved.

- *Unavailable appearance*: The button’s text and icons have an *embossed appearance*⁹, are drawn with 50% opacity, or are drawn such that only every other pixel is rendered in a checkerboard pattern.
- *Mixed appearance*: The button’s inner background is drawn—
 - such that the button face color and the button highlight color alternate every other pixel in a checkerboard pattern, or
 - as a solid color that’s a mixture of the button face color and the button highlight color.

The following is typical in buttons found in Windows versions 3.0 and 3.1¹⁰, Windows 95¹¹, and applications for these systems:

- For the pressed button style, the button’s text and icons are shifted one unit to the right and one unit down, compared to the unpressed style.

The following ways to draw buttons, default buttons, and toolbar buttons are typical in Windows 95¹² and applications for it:

- The mixed value style tends to be drawn as the unpressed variant, except the button’s text and icons have a *monochrome appearance* and its inner background has a *mixed appearance*.
- The unavailable style tends to be drawn as the unpressed variant, except the button’s text and icons have an *unavailable appearance*.
- For the option-set style, the button tends to be drawn as the normal pressed variant, except:
 - For the unpressed style, its inner background has a *mixed appearance*.¹³
 - For the unavailable style, its text and icons have an *unavailable appearance*.

In Presentation Manager and in System 7 of the Macintosh Operating System¹⁴, to render a button in the unavailable style, the entire button (including text, icons, and borders) is drawn such that only every other pixel is rendered in a “checkerboard” pattern.

Traditionally, the three-dimensional effects of buttons, icons, and other user-interface elements are based on a light source shining from the upper left.^{15 16}

⁹The *Windows Interface Guidelines for Software Design*, which applies to Windows 95.

¹⁰Buttons possessed a 3-D look (in that they appear to have depth or elevation) in Windows versions 3.0 and 3.1 by default, but not other interface elements. The article “Adding 3-D Effects to Controls” describes a library for these versions that gives 3-D appearances to more places in an application.

¹¹The *Windows Interface Guidelines for Software Design*, which applies to Windows 95.

¹²The *Windows Interface Guidelines for Software Design*, which applies to Windows 95.

¹³In this case, if the button is a toolbar button with a thin border, the button’s inner background involved in the mixed-value appearance has an inner outline colored in the button face color.

¹⁴*Macintosh Human Interface Guidelines*, p. 207 (according to which the system font in System 7 of the Macintosh Operating System was designed to be legible even when rendered this way).

¹⁵The *Windows Interface Guidelines for Software Design*, which applies to Windows 95.

¹⁶*Macintosh Human Interface Guidelines*, p. 232.

3.3 Button and Border Drawing Challenge

Here is a challenge. Write computer code (released to the public domain or licensed under the Unlicense) to draw the following ways to style borders and buttons:

- Window border, text box border, status field border, and grouping border.¹⁷
- The following elements in the unpressed, pressed, and unavailable (“disabled” or “inactive”) appearances:
 - Buttons in normal, “option-set”, and mixed-value (“indeterminate” or “third state”) states.
 - Buttons selected by default in normal, “option-set”, and mixed-value states.
 - Toolbar buttons in normal, “option-set”, and mixed-value states.
 - Checkboxes when unset (“unchecked” or “unticked”), when set (“checked” or “ticked”), and in mixed-value state.
 - Option buttons (“radio buttons”) when unset, option buttons when set, sliders.
- Toolbar buttons in normal, mixed-value, and “option-set” states: Hover style.
- Optionally, other user-interface elements (such as scroll bars and toggle switches).

Using only the following system colors and with some pixels or areas allowed to be transparent: the button face, window frame, button highlight, button shadow, button “light” highlight, and button dark shadow colors. It is allowed to simulate more colors using these six system colors by means of dithering.¹⁸

The border and button design should not employ trademarks, should be suitable for all ages, and must not involve the help of artificial-intelligence tools. Moreover, the design should not be a mere copy of border and button appearances found by default in Windows or other operating systems.

The *desktopwallpaper.py* file has some example code for border and button drawing.

4 Icons and Cursors

An icon (a small graphic representing a computer program, document, or resource) should come in a set of variations in color and dimensions:

- The same icon should be drawn in up to 2, up to 16, and up to 256 unique colors, and optionally with 8 bits per color component (also known as 8 bits per color channel or *8 bpc*). A traditional color choice for 16-color icons is the VGA palette.¹⁹

¹⁷Window borders are the outer edges of desktop windows. Text box borders are also known as “field borders”. Status field borders are the edges of inner boxes found in a *status bar*, which can appear on the bottom of some desktop windows. Grouping borders are the outer edges of areas that bring together several user-interface elements, such as checkboxes or option buttons (“radio buttons”) with a common purpose; grouping borders also serve as horizontal bars that separate parts of a menu.

¹⁸Dithering is the scattering of colors in a limited set to simulate colors outside that set.

¹⁹The VGA palette has 16 colors, each of which is one of the following: light gray, that is, (192, 192, 192); or each color component is 0 or 255; or each color component is 0 or 128. Windows CE before version 4.1 also supported four-color icons in a 2-bit-per-pixel format, where the colors tend to be the four gray tones of the VGA palette (namely, black or (0,0,0), white or (255,255,255), light gray, and dark gray or (128,128,128)) (“Display Buffer Formats”, Windows CE Device Driver Kit). A Windows color icon file can store an icon limited to 8 colors, but still in the 4-bit-per-pixel format, separately from 16-color icons in that format (Petzold, chapter 8), but such an 8-color icon is rarely seen in practice. Indeed, before version 3.0, the Windows EGA and VGA video drivers supported only eight logical colors rather than sixteen (Petzold, ch. 14), and there were no standard Windows color icon and bitmap file formats. (In addition, the Japanese computers PC-8801 and PC-9801 were equipped with eight-color video cards.) A traditional color choice for 8-color icons was a table of eight colors where each color component is 0 or 255. The EGA video driver for Windows version 3.1 supports 16 logical colors, but only 15 “physical” colors: the VGA palette is used, except the logical color light gray is missing and often replaced with a dithered mixture of dark gray and “white” (which is one possible way to adapt images colored using the VGA palette to the EGA driver). Moreover, the 16-color palette found in practice in Windows icons can differ slightly from the VGA palette. For example, in the Windows version 3.1 system palette, “the low-intensity system color values for the VGA [driver] are different from those for the [IBM] 8514 [video driver]” (Windows 3.1 SDK, Programmer’s Reference Volume 1, section 23.2.2), but how exactly they differ is not set forth in any article, as far as I am aware.

- The same icon should have dimensions 16×16 , 24×24 , 32×32 , 48×48 , and 64×64 pixels, and may be drawn in other dimensions to account for **pixel density**.²⁰
- Although the 256- and 16-color icons should be specially drawn if feasible, it is allowed to derive those icons from 8-bpc and 256-color icons, respectively, through an automated method.

Traditionally, 32×32 icons with the VGA palette are the most common variation.

Cursors (mouse pointer graphics) can follow the guidelines given earlier as well, but most cursors are traditionally in a single width and height, generally 32×32 pixels, except to account for **pixel density**.

Icons and cursors can include transparent pixels, but should have no translucent (semitransparent) pixels except for 8-bpc icons and cursors.

Note: Icon formats for OS/2 Presentation Manager and Microsoft Windows allow for icons and cursors with *inverted pixels* (where some existing pixels have their colors inverted), in addition to transparent and translucent (semitransparent) pixels. Describing these icon formats here is beyond the scope of this page, but see the **imageformat module documentation**²¹.

5 Drawing Style

In general, when user-interface graphics, including icons, cursors, and illustrations, are drawn using a limited number of colors, the following is observed. The observations apply to graphics from 1995 to about 2003, unless noted otherwise.

- Before 2003 (and even before 1995):
 - Curves and straight line segments are drawn without smoothing by approximating them with points at integer coordinates and coloring the pixels at those points.
 - Straight line segments are horizontal, are vertical, or have a slope equal to an integer or 1 divided by an integer. This can be achieved by drawing the line segment in equally sized steps.
 - There are no translucent (semitransparent) pixels.
- The three-dimensional (3-D) appearance of buttons and other objects in two-dimensional graphics supposes the presence of a light source shining from the upper left.²²
 - Graphics are drawn with a “black outline” on the bottom and right edges and with a “dark gray or other dark outline” on the other edges.^{23 24}
 - If a real-world object should have a 3-D look with a limited number of colors, that object is drawn in an *isometric* view (rather than straight on).²⁵ (This doesn’t apply to the edges of buttons.)
- Real-world objects depicted in user-interface graphics have an illustrative look with clean lines and curves rather than an abstract, pencil- or brush-drawn, highly realistic, or even *photorealistic* look.²⁶

²⁰Modern guidelines recommend a 256×256 icon as well. Toolbar icons are traditionally offered in 16×16 (or 16×15 ; see MFC Technical Note TN031, “Control Bars”) and 20×20 . The standard icon sizes in OS/2 Presentation Manager are 16×16 , 20×20 , 32×32 , and 40×40 (“Bitmap File Format”, in *Presentation Manager Programming Guide and Reference*); sometimes larger icons such as 64×64 occur. In *The Windows Interface Guidelines for Software Design* (Windows 95), the recommended icons were 16×16 in 16 colors, and 32×32 in 16 colors, and 48×48 in 256 colors (large version of desktop icon; see *Win32 Programmer’s Reference*).

²¹<https://github.com/peteroupc/classic-wallpaper/blob/main/imageformat.html>

²²*The Windows Interface Guidelines for Software Design*, which applies to Windows 95.

²³“**Creating Windows XP Icons**” (Microsoft Learn), July 2001. <https://learn.microsoft.com/en-us/previous-versions/microsoft/windows/xp/aa997636%28v=msdn.10%29>

²⁴Similar advice was also given in *The Microsoft Windows User Experience*. Before 1995 the icon outline tended to be black on all edges (see, for example, *Macintosh Human Interface Guidelines*, p. 239). And icons seen in Windows 3.0 (as opposed to version 3.1) tended to be drawn over a *drop shadow*, more specifically a dark gray silhouette of the icon, which silhouette is offset down and to the right by two units.

²⁵This is evident in the graphics (also known as *watermarks*) of Windows 95’s wizards, which are drawn in a teal background (color (0,128,128)) and show one or more computing devices in a three-dimensional, often rectangular appearance, and where, although there is internal shadowing, no shadow is cast on the teal background. But computer monitors may still be drawn straight on in order to accentuate what the monitor is showing.

²⁶Adventure games developed by Sierra On-Line in the early 1990s are well known to employ essentially simple lines and flood

- From 1990 to about 2003, for graphics limited to the 16-color VGA palette:²⁷
 - Areas are filled with either a solid color in the palette or an alternating checkerboard pattern of two colors (to simulate a color outside the palette).
 - Color gradient fills (smooth transitions from one color to another) and simulations of color gradients are rare (and then especially in backgrounds of illustrations), if not avoided.
- For graphics with 17 to 256 colors, gradient fills are present but subtle.
- Icons:
 - Before 1990, icons are limited to the two colors black and white, with optional transparency.²⁸
 - Before 1995, black-and-white icons (with no intermediate gray tones) from which a color version is derived do not use shading or hatch patterns to mimic shadows or solid colors.²⁹
 - Icons with more than 32 colors are rare; more than 64 colors, very rare.
 - Icons for toolbars, menu items, and the like do not behave like typographic symbols (dingbats), unlike the tendency in the late 2010s. For example, they are not designed in the same way as letters and digits in a typeface, or font; they can be colored; and they have less harmony with accompanying text than such symbols as the at-sign @.³⁰
 - Larger versions of originally 32×32 icons (for example, the 48×48 version) appear the same as the original icon but with finer but nonessential detail.³¹
- Cursors:
 - Most cursors are limited to the two colors black and white, with optional transparency.³²
 - Before 1990, cursors do not have colors other than black and white.
 - From 1990 to about 2003, cursors with colors other than black and white are rare, and cursors with more than 32 colors are very rare. From 1995 to about 2003, cursors with colors limited to the VGA palette become more common, especially infrequently seen cursors such as the busy-wait cursor.
 - Before 2003, and even before 1995, each shape of a black-and-white cursor is generally either white with a black outline (approximated with integer coordinates) or vice versa, to make the cursor easy to see over any background.

Note: After about 2003, user-interface graphics tend to be 8-bpc images (with or without translu-

fills in their illustrations. (A *flood fill* is a way to fill a colored area that is surrounded by other colors.) Windows 95 wizard watermarks are also of this style, essentially, except that the use of black outlines, as opposed to outlines of other colors, is rarer and less systematic.

²⁷The VGA palette has 16 colors, each of which is one of the following: light gray, that is, (192, 192, 192); or each color component is 0 or 255; or each color component is 0 or 128. Windows CE before version 4.1 also supported four-color icons in a 2-bit-per-pixel format, where the colors tend to be the four gray tones of the VGA palette (namely, black or (0,0,0), white or (255,255,255), light gray, and dark gray or (128,128,128)) (“Display Buffer Formats”, Windows CE Device Driver Kit). A Windows color icon file can store an icon limited to 8 colors, but still in the 4-bit-per-pixel format, separately from 16-color icons in that format (Petzold, chapter 8), but such an 8-color icon is rarely seen in practice. Indeed, before version 3.0, the Windows EGA and VGA video drivers supported only eight logical colors rather than sixteen (Petzold, ch. 14), and there were no standard Windows color icon and bitmap file formats. (In addition, the Japanese computers PC-8801 and PC-9801 were equipped with eight-color video cards.) A traditional color choice for 8-color icons was a table of eight colors where each color component is 0 or 255. The EGA video driver for Windows version 3.1 supports 16 logical colors, but only 15 “physical” colors: the VGA palette is used, except the logical color light gray is missing and often replaced with a dithered mixture of dark gray and “white” (which is one possible way to adapt images colored using the VGA palette to the EGA driver). Moreover, the 16-color palette found in practice in Windows icons can differ slightly from the VGA palette. For example, in the Windows version 3.1 system palette, “the low-intensity system color values for the VGA [driver] are different from those for the [IBM] 8514 [video driver]” (Windows 3.1 SDK, Programmer’s Reference Volume 1, section 23.2.2), but how exactly they differ is not set forth in any article, as far as I am aware.

²⁸The only icons and cursors supported by Windows versions before Windows 3.0 (in 1990) were limited to the two colors black and white.

²⁹*Macintosh Human Interface Guidelines*, p. 263.

³⁰Exceptions are found in the *Marlett*, *Wingdings*, and *Webdings* symbol fonts in Microsoft operating systems. <https://learn.microsoft.com/en-us/typography/font-list/marlett> <https://learn.microsoft.com/en-us/typography/font-list/wingdings> <https://learn.microsoft.com/en-us/typography/font-list/webdings>

³¹See also *Macintosh Human Interface Guidelines*, p. 233, which discusses deriving smaller icons from larger ones (in this case, 16×16 icons from 32×32 ones).

³²The only icons and cursors supported by Windows versions before Windows 3.0 (in 1990) were limited to the two colors black and white.

cent pixels) and are less interesting to discuss here, as 16- and 256-color versions are often made from those images through *dithering*³³ or similar techniques.

In general, from about 1990 to about 1997, user-interface text was drawn in one color only and rarely had smoothed edges; the edges were smoothed only if the display mode can show more than 256 colors at a time. In fancier ways to show text, a “shadowed” text look was often achieved using multiple shifted renderings of the text in a single color (for example, from one unit upward and leftward to three units downward and rightward) followed by an unshifted rendering in the base color or pattern.³⁴ But new applications should avoid having text in icons, cursors, and other images. An example of shadowed text follows:

The word "Sample" is rendered in a blue, serif font. It has a gray shadow offset slightly upwards and to the left, and a black shadow offset slightly downwards and to the right, creating a 3D effect.

Figure 1: The word “Sample” in blue, shadowed above by gray and below by black.

New user-interface graphics with limited colors ought to be designed as vector graphics (geometric models; for example, line segments and filled polygons) from the start, even if they are meant to resemble the drawing style given in this section when in their original size.³⁵ Existing images that function like icons should be **converted to vector graphics**³⁶ if they are simple enough.

6 Animations

Although Windows 95 and later versions have an *animation control* for displaying simple video files without sound that are limited to 256 colors, this control appears to be rarely used. More usually, traditional desktop applications don’t store an animation as a video file; rather, the images making up the animation are either stored as separate image files or arranged in a row or column of a single image file (in either case with transparent pixels marked with a color not present in the animation’s frames).³⁷ The source code file *desktopwallpaper.py* has a method, named `writeavi`, to write video files.

7 Flexible User Interface Graphics

For a high degree of flexibility, new graphical user interface systems should allow for the following:

- Designing icons, cursors, and other user-interface elements in the form of **vector graphics**³⁸ if desired.
- Having certain shapes in vector graphics be filled with system colors, the values of which are user-defined (such as a button face color or button highlight color), and drawing lines and curves in such graphics with system colors.
- Designing user-interface elements as images or vector graphics limited to gray tones, where the system replaces each gray tone in the image or graphic with the corresponding color in a color gradient involving one or more system colors.
- Drawing the same icon, cursor, or graphic—

³³*Dithering* is the scattering of colors in a limited set to simulate colors outside that set.

³⁴For example, see the discussion on buttons in the *RIPscrip* specification developed by TeleGrafix in 1992 and 1993. This specification was designed for building graphical user interfaces for online bulletin board systems under the EGA display mode.

³⁵I am reminded of the way textures were created for the modern game *QUOD* by “Daivuk”: they were stored as editing commands such as bevels, sunken areas, and tiling.

³⁶<https://peteroupc.github.io/classic-wallpaper/docs/pixeltovector.html>

³⁷The *Microsoft Windows User Experience* considers an animation to be fluid only if it runs at 16 or more frames per second. For an example of an animation in the form of a single image file, see Herman Rodent, “Animation in Win32”, Microsoft Developer Network, Feb. 1, 1994.

³⁸<https://peteroupc.github.io/classic-wallpaper/docs/pixeltovector.html>

- in multiple variations in size (width, height, or both), each with a different level of detail (where the system is expected to use a shrinking of the smallest available graphic that’s larger than the requested size, if the requested size is not available), even in the case of **vector graphics**^{39 40}, and
- with a different maximum number of unique colors (such as 2, 8, 16, 256, or 2^{24} colors).
- Animation of icons and cursors.

Given a graphic with multiple variations (such as in size, colors, or being a vector graphic), a system could then choose or synthesize the appropriate version of that graphic depending on the current display mode’s pixel density and which colors the mode can show. (For example, a vector graphic could be scaled up for high-pixel-density display modes, or a 256-color icon could be dithered to the VGA palette⁴¹ if the display mode can show only colors in that palette.)

³⁹<https://www.haiku-os.org/docs/userguide/en/applications/icon-o-matic.html>

⁴⁰For example, an image can be drawn with four variations in width and height: 32×32 , 40×40 , 32×24 , and 32×16 pixels. Multiple sizes and vector versions of a graphic are useful for several reasons, including: (1) to accommodate different display modes and pixel densities; (2) to render parts of the graphic more crisply, especially if their **smallest feature would measure less than twice the spacing between pixels**. They are useful for toolbar icons, for example, especially nowadays where the icon style is a single-color filled outline akin to a typographic symbol. Indeed, even 16×15 -pixel images often used as toolbar icons are, in many cases, ultimately vector graphics consisting of polygons and simple line segments.

⁴¹The VGA palette has 16 colors, each of which is one of the following: light gray, that is, (192, 192, 192); or each color component is 0 or 255; or each color component is 0 or 128. Windows CE before version 4.1 also supported four-color icons in a 2-bit-per-pixel format, where the colors tend to be the four gray tones of the VGA palette (namely, black or (0,0,0), white or (255,255,255), light gray, and dark gray or (128,128,128)) (“Display Buffer Formats”, Windows CE Device Driver Kit). A Windows color icon file can store an icon limited to 8 colors, but still in the 4-bit-per-pixel format, separately from 16-color icons in that format (Petzold, chapter 8), but such an 8-color icon is rarely seen in practice. Indeed, before version 3.0, the Windows EGA and VGA video drivers supported only eight logical colors rather than sixteen (Petzold, ch. 14), and there were no standard Windows color icon and bitmap file formats. (In addition, the Japanese computers PC-8801 and PC-9801 were equipped with eight-color video cards.) A traditional color choice for 8-color icons was a table of eight colors where each color component is 0 or 255. The EGA video driver for Windows version 3.1 supports 16 logical colors, but only 15 “physical” colors: the VGA palette is used, except the logical color light gray is missing and often replaced with a dithered mixture of dark gray and “white” (which is one possible way to adapt images colored using the VGA palette to the EGA driver). Moreover, the 16-color palette found in practice in Windows icons can differ slightly from the VGA palette. For example, in the Windows version 3.1 system palette, “the low-intensity system color values for the VGA [driver] are different from those for the [IBM] 8514 [video driver]” (Windows 3.1 SDK, Programmer’s Reference Volume 1, section 23.2.2), but how exactly they differ is not set forth in any article, as far as I am aware.

8 Relevant Works

There are many books and other works that discuss design matters on traditional user interfaces.^{42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66}

9 Worthy Mentions

- The `QLCDNumber` interface element, from the Qt framework, displays a number in a form resembling seven-segment displays. The number's digits are vector graphics (geometric models), not images, and `QLCDNumber` supports a drawing mode where the upper and left-hand outlines are drawn in a lighter color than the lower and right-hand outlines.
- The **Motif interface toolkit**⁶⁷ generates four kinds of system colors from a background color: a selection color, a foreground (text) color (which is either black or white), an upper shadow color, and a lower shadow color (generally darker than the upper shadow color), using an algorithm like the following that depends on the background color's calculated "brightness".⁶⁸ The **pseudocode conventions**⁶⁹ apply to the following pseudocode.

```
// First calculate the background color's "brightness",
// then calculate the derivative colors.
// Assumes each component of background color is
// from 0 through 1.
// 'getrgb' gets the color's three components.
```

⁴²*The Microsoft Windows User Experience*, which applies to Windows 98 and Windows 2000.

⁴³*The Windows Interface Guidelines for Software Design*, which applies to Windows 95.

⁴⁴"Wizard 97" (1999) and "Backward Compatible Wizard 97", part of the Windows Platform SDK, April 2000.

⁴⁵Matt Saettler, "Graphics Design and Optimization", Multimedia Technical Note (Microsoft), 1992.

⁴⁶W. Cherry and K. Marsh, "Adding 3-D Effects to Controls", Technical Note (Microsoft), 1992-1993.

⁴⁷Microsoft Knowledge Base article Q69079, "How to Give a 3-D Effect to Windows Controls".

⁴⁸K. Brockschmidt, "A Primer on Designing Custom Controls", *Microsoft Systems Journal* 7(2), March 1992.

⁴⁹K. P. Welch, "Extending the Windows 3.0 Interface with Installable Custom Controls", *Microsoft Systems Journal* 5(4), July 1990.

⁵⁰Kyle Marsh, "Implementing a Status Bar", Technical Note (Microsoft), April 2, 1992.

⁵¹Kyle Marsh, "Creating a Toolbar", Technical Note (Microsoft), December 31, 1992.

⁵²The "Visual Design Guide" that came with Microsoft Visual Basic 3.0 Professional Edition.

⁵³"Tip 53: Adding Three-Dimensional Effects to Visual Basic Controls", Visual Basic Tips, April 10, 1995.

⁵⁴*Common User Access: Basic Interface Design Guide* and *Common User Access: Advanced Interface Design Guide*, which apply to Windows version 3.0 and Presentation Manager.

⁵⁵Shiz Kobara, *Visual Design with OSF/Motif*, Addison-Wesley, 1991.

⁵⁶*OSF/Motif Style Guide* (releases 1.1, 1.2, and 2.0), and *OSF/Motif Widget Writer's Guide*.

⁵⁷*The Windows Interface: An Application Design Guide*, which applies to Windows version 3.1.

⁵⁸*Motif Reference Manual* (Volume Six B) and *XView Reference Manual* (Volume Seven B), from the X Window System series published by O'Reilly & Associates.

⁵⁹*Macintosh Human Interface Guidelines*, 1992. <https://doi.org/10.5555/573097>

⁶⁰*Mac OS 8 Human Interface Guidelines* (addendum to *Macintosh User Interface Guidelines*), Sep. 2, 1997.

⁶¹"Color, Windows and 7.0", Apple Technical Note TB33, Oct. 1, 1992.

⁶²E. Voas, "Appearance: Not Just Another Pretty Interface", *develop* (Apple), June 1997.

⁶³J. Osborne, D. Thomas, "Working in the Third Dimension", *develop* (Apple), September 1993, describes the authors' suggestions for the three-dimensional appearance of buttons and certain other interface elements compatible with System 7 of the Macintosh Operating System.

⁶⁴"Creating Windows XP Icons" (Microsoft Learn), July 2001. <https://learn.microsoft.com/en-us/previous-versions/m-s997636%28v=msdn.10%29>

⁶⁵Commodore-Amiga, Inc., *Amiga User Interface Style Guide*, Addison-Wesley, 1991. Suggested to me by a Reddit community.

⁶⁶C. Petzold, *Programming Windows: The Microsoft Guide to Writing Applications for Windows 3*, Microsoft Press, 1990.

⁶⁷<https://github.com/fjardon/motif>

⁶⁸The resulting color may vary slightly from the one calculated by the Motif toolkit, because of rounding errors committed by that toolkit.

⁶⁹<https://peteroupc.github.io/pseudocode.html>

```

rgb=getrgb(background); r=rgb[0];g=rgb[1];b=rgb[2]
// default values for thresholds
foregroundThreshold=0.7
lightThreshold=0.93
darkThreshold=0.2
// find "brightness" of background color
brightness=0.75 * ((r+g+b)/3) + 0.25 * (0.3*r+0.59*g+0.11*b)
// find foreground color
if brightness>foregroundThreshold
    foreground=[0,0,0] // black
else
    foreground=[1,1,1] // white
end
if brightness<darkThreshold
    // very dark color
    select_color=background+0.15*(1-background)
    lower_shadow=background+0.3*(1-background)
    upper_shadow=background+0.5*(1-background)
else if brightness>lightThreshold
    // very light color
    select_color=background-0.15*background
    lower_shadow=background-0.4*background
    upper_shadow=background-0.2*background
else
    // medium color
    select_color=background-0.15*background
    fac = 0.6-0.2*brightness
    lower_shadow=background-fac*background
    fac = 0.5+0.1*brightness
    upper_shadow=background+fac*(1-background)
end

```

10 License

Any copyright to this page is released to the Public Domain. In case this is not possible, this page is also licensed under the **Unlicense**⁷⁰.

11 End Notes

⁷⁰<https://unlicense.org>